Consider the following code segment.

```
#include <iostream>
using namespace std;

class B {
    char *p;
public:
    B(const char *s) : p(const_cast<char*>(s)) { }
    const char* show() const { return p; }
    void change(const char *q) { p = const_cast<char*>(q); }
};

int main() {
    const char *lit = "Hello";
    char *y = const_cast<char*>(lit);   // LINE-1
    const B u(lit);
    u.change(lit);                      // LINE-2
    cout << u.show();                   // LINE-3
    const_cast<B&>(u).change(y);        // LINE-4
    return 0;
}
```

Which line/s will give error?

a) LINE-1

b) LINE-2

c) LINE-3

d) LINE-4

Consider the following code segment.

```
class X { };
class Y { };
X* px = new X;
Y* py = new Y;
```

Which of the following type-castings is **permissible** (i.e. compiles)?

a) `py = static_cast<Y*>(px);`

b) `py = dynamic_cast<Y*>(px);`

c) `py = reinterpret_cast<Y*>(px);`

d) `py = const_cast<Y*>(px);`

Consider the following code segment.

```
#include <iostream>
using namespace std;

class Parent {
public:
    virtual void show() {
        cout << "Parent ";
    }
};

class Child : public Parent {
public:
    virtual void show() {
        cout << "Child ";
    }
};

int main() {
    Parent p;
    Child c;
```

```
    try {
        Parent& r1 = static_cast<Parent&>(c);   // LINE-1
        r1.show();
        Parent& r2 = dynamic_cast<Parent&>(c);  // LINE-2
        r2.show();
        Child& r3 = static_cast<Child&>(p);     // LINE-3
        r3.show();
        Child& r4 = dynamic_cast<Child&>(p);    // LINE-4
        r4.show();
    }
    catch (exception& e) {
        cout << e.what();
    }
    return 0;
}
```

What will be the output?

a) `Child Child Parent`

b) `Child Child Parent Parent`

c) `Child Child Parent std::bad_cast`

d) `Child Parent Child Parent`

Consider the code segment given below.

```
#include <iostream>
using namespace std;
class X { public: virtual ~X() {} };
class Y : public X {};
class Z : public X {};
int main(){
    X objX;
    Y objY;
    X* p1 = dynamic_cast<X*>(&objY); //LINE-1
    p1 == NULL ? cout << "10 " : cout << "11 ";
    Y* p2 = dynamic_cast<Y*>(p1); //LINE-2
    p2 == NULL ? cout << "20 " : cout << "21 ";
    Z* p3 = dynamic_cast<Z*>(new X); //LINE-3
    p3 == NULL ? cout << "30 " : cout << "31 ";
    Z* p4 = dynamic_cast<Z*>(&objY); //LINE-4
    p4 == NULL ? cout << "40 " : cout << "41 ";
    return 0;
}
```

What will be the output?

a) 11 21 31 40

b) 11 21 30 40

c) 11 20 31 40

d) 11 21 31 41

Consider the following code segment.

```
#include <iostream>
using namespace std;

class A {
public:
    virtual void f() {}
    virtual void g() {}
};

class B : public A {
public:
    void f() {}
    void g() {}
    virtual void j() {}
};

class C : public B {
public:
    void f()  {}
    virtual void h() {}
};
```

What will be the virtual function table (VFT) for the `class C`?

a)
```
C::f(C* const)
B::g(B* const)
C::h(C* const)
B::j(B* const)
```

b)
```
A::f(A* const)
B::g(B* const)
C::h(C* const)
B::j(B* const)
```

c)
```
C::f(C* const)
A::g(A* const)
C::h(C* const)
C::j(C* const)
```

d)
```
C::f(C* const)
B::g(B* const)
B::h(B* const)
B::j(B* const)
```

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    float f = 5.25;
    int *px = &x;
    float *pf;
    x = static_cast<int>(f);       // LINE-1
    f = static_cast<float>(x);     // LINE-2
    pf = static_cast<float*>(px); // LINE-3
    x = static_cast<int>(&x);      // LINE-4
    return 0;
}
```

Which line/s will give compilation error?

a) LINE-1

b) LINE-2

c) LINE-3

d) LINE-4

Consider the code segment given below.

```
class Vehicle {
public:
    virtual void start() = 0;
    void stop() {};
};
class Car: public Vehicle {
public:
    void start() {};
    virtual void honk() {};
};
class Bike: public Vehicle {
public:
    void start() {};
    virtual void wheelie() {};
};
class Truck: public Car {
public:
    void honk() {};
    virtual void loadCargo() {};
};
class Bicycle {
public:
    void pedal() {};
    void brake() {};
};
```

How many virtual function tables (VFT) will be created?

a) 3

b) 4

c) 5

d) 6

Consider the following code segment.

```
#include <iostream>
#include <typeinfo>
using namespace std;

class B {
public:
    virtual ~B() {}
};

class D : public B {};

int main() {
    B bobj;
    D dobj;
    B *bp1 = &dobj;
    B *bp2 = &bobj;
    D *dp  = &dobj;

    cout << (typeid(bp1) == typeid(bp2));
    cout << (typeid(*bp1) == typeid(*dp));
    cout << (typeid(*bp2) == typeid(*dp));
    cout << (typeid(dp) == typeid(bp1));

    return 0;
}
```

What will be the output?

a) 1001

b) 1100

c) 0101

d) 0110

Consider the code snippet below.

```
#include <iostream>
using namespace std;

class Course{
    string _name;
    int _credits;
public:
    Course(string name, int credits)
        : _name(name), _credits(credits) {}

    void updateCredits(int c) const {
        (______________)->_credits = c;   // LINE-1
    }

    void show() const {
        cout << _name << ":" << _credits;
    }
};

int main(){
    const Course c("OOP in C++", 3);
    c.updateCredits(4);
    c.show();
    return 0;
}
```

Fill in the blank at `LINE-1` so that the output becomes: `OOP in C++:4`

a) `(Course*)this`

b) `const_cast<Course*>(this)`

c) `static_cast<Course*>(this)`

d) `dynamic_cast<Course*>(this)`